

Natural Language Processing

04. Modèles de langues probabilistes

Kenza BOUZIDI

`kenza.bouzidi.ext@u-pec.fr`

19 Décembre 2025

- La problématique du *language modeling*
- Les modèles *trigram*
- Évaluer les *language models* : la *perplexity*
- Techniques d'estimation :
 1. *Linear interpolation*
 2. Méthodes de *discounting*

Le Language Modeling : problématique

- *Language Model* : une fonction qui assigne des probabilités à n'importe quelle phrase
- *Good language model* : le modèle assigne des probabilités plus élevées aux phrases qui sont naturelles et qui ont du sens

Pourquoi ?



Pourquoi voudrait-on faire cela ?

- La *speech recognition* a été la motivation originale.
- Considérons les phrases : 1) *recognize speech* et 2) *wreck a nice beach*.
- Ces deux phrases sonnent très similaires lorsqu'elles sont prononcées, ce qui rend difficile pour un système automatique de *speech recognition* de les transcrire correctement.
- Lorsque le système analyse l'audio et tente une transcription, il prend en compte les probabilités du *language model* pour déterminer l'interprétation la plus probable.
- Le *language model* favorisera $p(\text{recognize speech})$ plutôt que $p(\text{wreck a nice beach})$.
- Car la première est une phrase plus courante et devrait apparaître plus fréquemment dans le corpus d'entraînement.

Pourquoi voudrait-on faire cela ?

- En incorporant des *language models*, les systèmes de *speech recognition* peuvent améliorer leur précision en sélectionnant la phrase qui correspond le mieux aux patrons linguistiques et au contexte, même face à des alternatives très similaires au niveau sonore.
- Des problèmes liés incluent l'*optical character recognition* (la reconnaissance d'écriture manuscrite).
- En fait, les *Language Models* sont utiles dans toutes les tâches de NLP impliquant la génération de langage (par ex., *machine translation*, *summarization*, *chatbots*).
- Les techniques d'estimation développées pour ce problème seront TRÈS utiles pour d'autres problèmes en NLP.

Le Language Modeling : problématique

- Nous avons un vocabulaire (fini), disons $\mathcal{V} = \{\text{the, a, man, telescope, Beckham, two, . . .}\}$
- Nous avons un ensemble (infini) de chaînes, $\mathcal{V}^*$.
- Par exemple :
 - the STOP
 - a STOP
 - the fan STOP
 - the fan saw Beckham STOP
 - the fan saw saw STOP
 - the fan saw Beckham play for Real Madrid STOP
- Où STOP est un symbole spécial indiquant la fin d'une phrase.

Le Language Modeling : problématique (suite)

- Nous avons un échantillon d'entraînement contenant des phrases d'exemple en anglais.
- Nous devons apprendre une distribution de probabilité p .
- p est une fonction qui satisfait :

$$\sum_{x \in V^*} p(x) = 1$$

$$p(x) \geq 0 \quad \text{pour tout } x \in V^*$$

- Exemples de probabilités assignées à des phrases :

$$p(\text{the STOP}) = 10^{-12}$$

$$p(\text{the fan STOP}) = 10^{-8}$$

$$p(\text{the fan saw Beckham STOP}) = 2 \times 10^{-8}$$

$$p(\text{the fan saw saw STOP}) = 10^{-15}$$

...

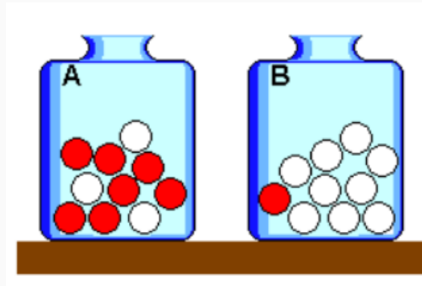
$$p(\text{the fan saw Beckham play for Real Madrid STOP}) = 2 \times 10^{-9}$$

Le Language Modeling : problématique (suite)

- Idée 1 : Le modèle assigne une probabilité plus élevée aux phrases *fluent* (celles qui ont du sens et sont grammaticalement correctes).
- Idée 2 : Estimer cette fonction de probabilité à partir du texte (corpus).
- Le *language model* aide les modèles de génération de texte à distinguer entre les bonnes et les mauvaises phrases.

Les Language Models sont génératifs

- Les *language models* peuvent générer des phrases en échantillonnant séquentiellement à partir des probabilités.
- C'est analogue à tirer des boules (mots) d'une urne où leur taille est proportionnelle à leur fréquence relative.
- Alternativement, on pourrait toujours choisir le mot le plus probable, ce qui revient à prédire le mot suivant.



Une méthode naïve

- Une méthode très naïve pour estimer la probabilité d'une phrase consiste à compter le nombre d'occurrences de la phrase dans les données d'entraînement et à diviser par le nombre total de phrases d'entraînement (N) pour obtenir une estimation de la probabilité.
- Nous avons N phrases d'entraînement.
- Pour toute phrase $x_1, x_2, \dots, x_n$, $c(x_1, x_2, \dots, x_n)$ est le nombre de fois où la phrase apparaît dans les données d'entraînement.
- Une estimation naïve :

$$p(x_1, x_2, \dots, x_n) = \frac{c(x_1, x_2, \dots, x_n)}{N}$$

- Problème : Comme le nombre de phrases possibles croît exponentiellement avec la longueur de la phrase et la taille du vocabulaire, il devient de plus en plus improbable qu'une phrase spécifique apparaisse dans les données d'entraînement.
- Par conséquent, beaucoup de phrases auront une probabilité nulle selon le modèle naïf, ce qui entraîne une très mauvaise généralisation.

Modèles Trigram

- Considérons une séquence de variables aléatoires $X_1, X_2, \dots, X_n$.
- Chaque variable aléatoire peut prendre n'importe quelle valeur dans un ensemble fini V .
- Pour l'instant, nous supposons que la longueur n est fixe (par ex., $n = 100$).
- Notre objectif : modéliser $P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n)$

Processus de Markov d'ordre 1

$$\begin{aligned}P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) &= P(X_1 = x_1) \prod_{i=2}^n P(X_i = x_i | X_1 = x_1, \dots, X_{i-1} = x_{i-1}) \\&= P(X_1 = x_1) \prod_{i=2}^n P(X_i = x_i | X_{i-1} = x_{i-1})\end{aligned}$$

L'hypothèse de Markov d'ordre 1 : Pour tout $i \in \{2, \dots, n\}$ et tout $x_1, \dots, x_i$,

$$P(X_i = x_i | X_1 = x_1, \dots, X_{i-1} = x_{i-1}) = P(X_i = x_i | X_{i-1} = x_{i-1})$$

Processus de Markov d'ordre 2

$$\begin{aligned} P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) &= \\ P(X_1 = x_1) \cdot P(X_2 = x_2 | X_1 = x_1) \cdot \prod_{i=3}^n P(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1}) \\ &= \prod_{i=1}^n P(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1}) \end{aligned}$$

(Pour simplifier, on suppose que $x_0 = x_{-1} = *$, où $*$ est un symbole spécial de "start".)

Modélisation de séquences de longueur variable

- Nous aimerions que la longueur de la séquence, n , soit également une variable aléatoire.
- Une solution simple : définir toujours $X_n = \text{STOP}$, où STOP est un symbole spécial.
- Puis utiliser un processus de Markov comme précédemment :

$$P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = \prod_{i=1}^n P(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

- (Pour simplifier, on suppose que $x_0 = x_{-1} = *$, où $*$ est un symbole spécial de "start".)

Modèles de langage Trigram

- Un *trigram language model* consiste en :
 1. Un ensemble fini V
 2. Un paramètre $q(w|u, v)$ pour chaque trigramme u, v, w tel que $w \in V \cup \{\text{STOP}\}$ et $u, v \in V \cup \{*\}$
- Pour toute phrase $x_1 \dots x_n$ où $x_i \in V$ pour $i = 1 \dots (n-1)$ et $x_n = \text{STOP}$, la probabilité de la phrase selon le modèle trigram est :

$$p(x_1 \dots x_n) = \prod_{i=1}^n q(x_i | x_{i-2}, x_{i-1})$$

- On définit $x_0 = x_{-1} = *$ pour simplifier les notations.

Un exemple

Pour la phrase the dog barks STOP, nous aurions :

$$p(\text{the dog barks STOP}) = q(\text{the}|\ast, \ast) \times q(\text{dog}|\ast, \text{the}) \times q(\text{barks}|\text{the}, \text{dog}) \times q(\text{STOP}|\text{dog}, \text{barks})$$

Le problème d'estimation pour les Trigram

Problème d'estimation restant :

$$q(w_i | w_{i-2}, w_{i-1})$$

Par exemple :

$$q(\text{laughs} | \text{the}, \text{dog})$$

Une estimation naturelle (l' estimation du maximum de vraisemblance) :

$$q(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$$

Par exemple :

$$q(\text{laughs} | \text{the}, \text{dog}) = \frac{\text{Count}(\text{the}, \text{dog}, \text{laughs})}{\text{Count}(\text{the}, \text{dog})}$$

Une estimation naturelle (l' estimation du maximum de vraisemblance) :

$$q(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$$

$$q(\text{laughs} | \text{the}, \text{dog}) = \frac{\text{Count}(\text{the}, \text{dog}, \text{laughs})}{\text{Count}(\text{the}, \text{dog})}$$

- Supposons que la taille de notre vocabulaire soit $N = |V|$, alors il y a N^3 paramètres dans le modèle.
- Par exemple, $N = 20,000 \Rightarrow 20,000^3 = 8 \times 10^{12}$ paramètres.

Perplexity

Évaluer un Language Model : Perplexity

- Nous avons des données de test, m phrases : $s_1, s_2, s_3, \dots, s_m$
- On pourrait regarder la probabilité selon notre modèle : $\prod_{i=1}^m p(s_i)$. Ou, plus simplement, la log-probabilité :

$$\log \left(\prod_{i=1}^m p(s_i) \right) = \sum_{i=1}^m \log p(s_i)$$

- En pratique, la mesure d'évaluation habituelle est la *perplexity* :

$$\text{Perplexity} = 2^{-I} \quad \text{où} \quad I = \frac{1}{M} \sum_{i=1}^m \log p(s_i)$$

- M est le nombre total de mots dans les données de test

Quelques intuitions sur la Perplexity

- Supposons que nous ayons un vocabulaire V , et $N = |V| + 1$, et un modèle qui prédit :

$$q(w|u, v) = \frac{1}{N} \quad \text{pour tout } w \in V \cup \{\text{STOP}\}, \text{ pour tout } u, v \in V \cup \{*\}$$

- Il est facile de calculer la perplexity dans ce cas :

$$\text{Perplexity} = 2^{-I} \quad \text{où} \quad I = \log \frac{1}{N} \Rightarrow \text{Perplexity} = N$$

- La perplexity peut être vue comme une mesure du branching factor effectif

Quelques intuitions sur la Perplexity

- **Preuve :** Supposons que nous ayons m phrases de longueur n dans le corpus, et M le nombre total de tokens dans le corpus, $M = m \cdot n$.
- Considérons la log-probabilité (base 2) d'une phrase $s = w_1 w_2 \dots w_n$ selon le modèle :

$$\log p(s) = \log \prod_{i=1}^n q(w_i | w_{i-2}, w_{i-1}) = \sum_{i=1}^n \log q(w_i | w_{i-2}, w_{i-1})$$

- Comme chaque $q(w_i | w_{i-2}, w_{i-1}) = \frac{1}{N}$, nous avons :

$$\log p(s) = \sum_{i=1}^n \log \frac{1}{N} = n \cdot \log \frac{1}{N} = -n \cdot \log N$$

$$I = \frac{1}{M} \sum_{i=1}^m \log p(s_i) = \frac{1}{M} \sum_{i=1}^m -n \cdot \log N = \frac{1}{M} \cdot -m \cdot n \cdot \log N = -\log N$$

- Par conséquent, la perplexity est donnée par :

$$\text{Perplexity} = 2^{-I} = 2^{-(-\log N)} = N$$

Valeurs typiques de Perplexity

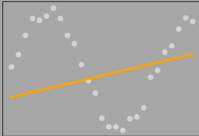
- Résultats de Goodman ("A bit of progress in language modeling"), avec $|V| = 50,000$ [Goodman, 2001].
- Un modèle trigram : $p(x_1, \dots, x_n) = \prod_{i=1}^n q(x_i | x_{i-2}, x_{i-1})$
Perplexity = 74
- Un modèle bigram : $p(x_1, \dots, x_n) = \prod_{i=1}^n q(x_i | x_{i-1})$
Perplexity = 137
- Un modèle unigram : $p(x_1, \dots, x_n) = \prod_{i=1}^n q(x_i)$
Perplexity = 955

Smoothing

Le compromis Bias-Variance

Bias-Variance Tradeoff

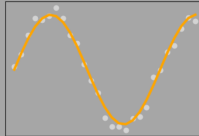
High Bias / Low Variance



Underfitting

The model is too simple and not able to identify the underlying pattern of the data

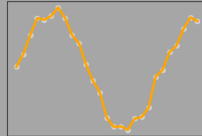
Optimal Bias



Optimal fit

The model fits the data well and identifies the underlying pattern.

Low Bias / High Variance



Overfitting

The model is too complex and fits the training data too well. It is not able to identify the underlying pattern.

Datamapu

¹Source de l'image : *Datamapu* - Bias and Variance

- Estimation trigram du maximum de vraisemblance :

$$q_{\text{ML}}(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$$

- Estimation bigram du maximum de vraisemblance :

$$q_{\text{ML}}(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

- Estimation unigram du maximum de vraisemblance :

$$q_{\text{ML}}(w_i) = \frac{\text{Count}(w_i)}{\text{Count}()}$$

- On prend notre estimation $q(w_i|w_{i-2}, w_{i-1})$ comme :

$$q(w_i|w_{i-2}, w_{i-1}) = \lambda_1 \cdot q_{\text{ML}}(w_i|w_{i-2}, w_{i-1}) + \lambda_2 \cdot q_{\text{ML}}(w_i|w_{i-1}) + \lambda_3 \cdot q_{\text{ML}}(w_i)$$

avec $\lambda_1 + \lambda_2 + \lambda_3 = 1$, et $\lambda_i \geq 0$ pour tout i .

- Notre estimation définit correctement une distribution (on définit $V' = V \cup \{\text{STOP}\}$) :

$$\begin{aligned} & \sum_{w \in V'} q(w|u, v) \\ &= \sum_{w \in V'} [\lambda_1 \cdot q_{\text{ML}}(w|u, v) + \lambda_2 \cdot q_{\text{ML}}(w|v) + \lambda_3 \cdot q_{\text{ML}}(w)] \\ &= \lambda_1 \sum_w q_{\text{ML}}(w|u, v) + \lambda_2 \sum_w q_{\text{ML}}(w|v) + \lambda_3 \sum_w q_{\text{ML}}(w) \\ &= \lambda_1 + \lambda_2 + \lambda_3 = 1 \end{aligned}$$

- On peut également montrer que $q(w|u, v) \geq 0$ pour tout $w \in V'$.

- On réserve une partie du jeu d'entraînement comme données de *validation*.
- On note $c'(w_1, w_2, w_3)$ le nombre de fois où le trigramme (w_1, w_2, w_3) apparaît dans le jeu de validation.
- On choisit $\lambda_1, \lambda_2, \lambda_3$ pour maximiser :

$$L(\lambda_1, \lambda_2, \lambda_3) = \sum_{w_1, w_2, w_3} c'(w_1, w_2, w_3) \log q(w_3 | w_1, w_2)$$

sous la contrainte $\lambda_1 + \lambda_2 + \lambda_3 = 1$, et $\lambda_i \geq 0$ pour tout i , et où

$$q(w_i | w_{i-2}, w_{i-1}) = \lambda_1 \cdot q_{\text{ML}}(w_i | w_{i-2}, w_{i-1}) + \lambda_2 \cdot q_{\text{ML}}(w_i | w_{i-1}) + \lambda_3 \cdot q_{\text{ML}}(w_i)$$

Méthodes de Discounting

- Considérons les comptes suivants et les estimations du maximum de vraisemblance :

Sentence	Count	$q_{\text{ML}}(w_i w_{i-1})$
the	48	
the, dog	15	15/48
the, woman	11	11/48
the, man	10	10/48
the, park	5	5/48
the, job	2	2/48
the, telescope	1	1/48
the, manual	1	1/48
the, afternoon	1	1/48
the, country	1	1/48
the, street	1	1/48

- Les estimations du maximum de vraisemblance sont élevées, particulièrement pour les items à faible fréquence.

Méthodes de Discounting

- On définit les comptes discounted comme suit :

$$\text{Count}^*(x) = \text{Count}(x) - 0.5$$

Sentence	Count	Count*(x)	$q_{\text{ML}}(\mathbf{w}_i \mathbf{w}_{i-1})$
the	48		
the, dog	15	14.5	14.5/48
the, woman	11	10.5	10.5/48
the, man	10	9.5	9.5/48
the, park	5	4.5	4.5/48
the, job	2	1.5	1.5/48
the, telescope	1	0.5	0.5/48
the, manual	1	0.5	0.5/48
the, afternoon	1	0.5	0.5/48
the, country	1	0.5	0.5/48
the, street	1	0.5	0.5/48

- Les nouvelles estimations sont basées sur ces comptes discounted .

Méthodes de Discounting (suite)

- Nous avons maintenant une missing probability mass :

$$\alpha(w_{i-1}) = 1 - \sum_w \frac{\text{Count}^*(w_{i-1}, w)}{\text{Count}(w_{i-1})}$$

- Par exemple, dans notre cas :

$$\alpha(\text{the}) = \frac{10 \times 0.5}{48} = \frac{5}{48}$$

Modèles Katz Back-Off (Bigrams)

- Pour un modèle bigram, on définit deux ensembles :

$$A(w_{i-1}) = \{w : \text{Count}(w_{i-1}, w) > 0\}$$

$$B(w_{i-1}) = \{w : \text{Count}(w_{i-1}, w) = 0\}$$

- Un modèle bigram :

$$q_{\text{BO}}(w_i | w_{i-1}) = \begin{cases} \frac{\text{Count}^*(w_{i-1}, w_i)}{\text{Count}(w_{i-1})} & \text{si } w_i \in A(w_{i-1}) \\ \frac{\alpha(w_{i-1}) q_{\text{ML}}(w_i)}{\sum_{w \in B(w_{i-1})} q_{\text{ML}}(w)} & \text{si } w_i \in B(w_{i-1}) \end{cases}$$

- Où :

$$\alpha(w_{i-1}) = 1 - \sum_{w \in A(w_{i-1})} \frac{\text{Count}^*(w_{i-1}, w)}{\text{Count}(w_{i-1})}$$

Modèles Katz Back-Off (Trigrams)

- Pour un modèle trigram, on définit d'abord deux ensembles :

$$A(w_{i-2}, w_{i-1}) = \{w : \text{Count}(w_{i-2}, w_{i-1}, w) > 0\}$$

$$B(w_{i-2}, w_{i-1}) = \{w : \text{Count}(w_{i-2}, w_{i-1}, w) = 0\}$$

- Un modèle trigram est défini en termes du modèle bigram :

$$q_{\text{BO}}(w_i | w_{i-2}, w_{i-1}) = \begin{cases} \frac{\text{Count}^*(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})} & \text{si } w_i \in A(w_{i-2}, w_{i-1}) \\ \frac{\alpha(w_{i-2}, w_{i-1}) q_{\text{BO}}(w_i | w_{i-1})}{\sum_{w \in B(w_{i-2}, w_{i-1})} q_{\text{BO}}(w | w_{i-1})} & \text{si } w_i \in B(w_{i-2}, w_{i-1}) \end{cases}$$

- Où :

$$\alpha(w_{i-2}, w_{i-1}) = 1 - \sum_{w \in A(w_{i-2}, w_{i-1})} \frac{\text{Count}^*(w_{i-2}, w_{i-1}, w)}{\text{Count}(w_{i-2}, w_{i-1})}$$

- Dériver les probabilités dans les probabilistic language models implique trois étapes :
 1. Développer $p(w_1, w_2, \dots, w_n)$ en utilisant la Chain rule.
 2. Appliquer les Markov Independence Assumptions
$$p(w_i | w_1, w_2, \dots, w_{i-2}, w_{i-1}) = p(w_i | w_{i-2}, w_{i-1}).$$
 3. Lisser les estimations en utilisant les low order counts.
- D'autres méthodes pour améliorer les language models incluent :
 - Introduire des latent variables pour représenter des topics, connus sous le nom de topic models. [Blei et al., 2003]
 - Remplacer $p(w_i | w_1, w_2, \dots, w_{i-2}, w_{i-1})$ par un predictive neural network et une "embedding layer" pour mieux représenter de plus grands contextes et exploiter les similarités entre les mots du contexte. [Bengio et al., 2000]
- Les modern language models utilisent des deep neural networks dans leur architecture et disposent d'un espace de paramètres très vaste.

-  Bengio, Y., Ducharme, R., & Vincent, P. (2000). *A neural probabilistic language model*. Advances in Neural Information Processing Systems, 13.
-  Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). *Latent Dirichlet Allocation*. Journal of Machine Learning Research, 3(Jan), 993–1022.
-  Goodman, J. T. (2001). *A bit of progress in language modeling*. Computer Speech & Language, 15(4), 403–434.